

CS342 OPERATING SYSTEMS

Spring 2026 - Project #1

PROCESSES, IPC, and THREADS

Assigned: Feb 6, 2026.

Due date: Feb 22, 2026; 23:59.

Document Version: 1.0

- You will develop the project in C/Linux. Submitted projects will be tested in **Ubuntu 24.04 Linux 64-bit**.
 - You are not allowed to share this project and/or its solution [1].
 - **Objectives/keywords:** Learn and practice with process creation, fork, inter-process communication, threads, multi-process applications, multi-threaded applications.
-

PART A (50 pts): DEVELOPING A MULTIPROCESS APPLICATION

In this project, you will develop a multi-process application that reads a set of input files and reports information about long words found in those files. The program will use multiple child processes to perform this task [2,3]. The program will be called **findlwp** (where “p” stands for processes), and the corresponding C source file will be named **findlwp.c**. The program will be invoked with the following command-line parameters.

findlwp FilePrefix N K DataLen OutFilename

N specifies the number of input files. The program processes these files and identifies all words whose length is greater than or equal to K. The input files are named using the format **FilePrefix[1..N]**. For example, if N = 3 and FilePrefix is "infile", the input files will be named "infile1", "infile2", and "infile3".

Each input file is an ASCII text file containing words. A word is defined as a maximal sequence of non-whitespace characters delimited by one or more whitespace characters (SPACE, TAB, or NEWLINE).

The parameter K defines the minimum word-length threshold; any word whose length is greater than or equal to K is considered. The DataLen parameter specifies the maximum number of bytes that can be transferred in a single write() or read() system call on a POSIX pipe. The parameter OutFilename specifies the name of the output file.

The program will create N child processes, one child process per input file. It will also create N pipes, one pipe per child process, allowing each child to transmit its results to the parent process [4]. Each child process is responsible

for processing a distinct input file. Note that you should create a pipe before creating a child process so that the pipe descriptors can be passed to the child.

A child process reads its assigned input file and extracts all words whose length is greater than or equal to K. For each such word, the child records (in an appropriate data structure) the line number of every occurrence of that word. If a word appears multiple times on the same line, the corresponding line number is recorded multiple times. Line numbering will start from 1.

After fully processing the input file, the child process sends the collected information to the parent process via its pipe (child-to-parent communication). Each child process performs this task independently and runs concurrently with the other child processes. After transmitting its data, the child process terminates.

The parent process receives data from all child processes through the pipes, merges the received information, sorts the information, and writes the results to the output file specified by OutFilename. The parent process executes concurrently with the child processes and must read from the pipes while data is being produced (i.e., is being sent to pipes). The parent should avoid blocking unnecessarily on a pipe when data may be available on another pipe (e.g., by using appropriate I/O multiplexing mechanisms). After processing all incoming data and writing the output, the parent process terminates.

The format of the output file is defined as follows:

```
wordx (count=3): 1-34, 1-67, 3-15
wordyz (count=4): 2-76, 2-102, 3-102, 3-145
...
```

This output indicates that the word "wordx" appears three times across the input files: twice in file 1, at lines 34 and 67, and once in file 3, at line 15.

Sorting should be performed with respect to words (should be case-sensitive), in increasing (lexicographical) order. You should use strcmp() to compare two words and determine their relative order. For each word, the appearance information should be sorted first by file index and then by line number, both in increasing order. Count information should not be considered during sorting.

Each child process sends its data to the parent through its pipe in chunks of at most DataLen bytes per write() system call. Similarly, the parent process reads from each pipe in chunks of at most DataLen bytes per read() system call.

Example invocations of the program are shown below:

```
./findlwp infile 4 7 1 out.txt
```

```
./findlwp input 10 6 100 out2.txt
```

Please note the following specifications:

- The minimum value of DataLen can be 1.
- The maximum value of DataLen can be 1000.
- The minimum value of N can be 1.
- The maximum value of N can be 10.
- The minimum value of K can be 1.
- The maximum value of K can be 100.

In the parent process, to read from multiple pipes concurrently, you may use non-blocking I/O, or you may use the select() system call to determine which pipe(s) have data available to be read.

The pipe() function creates a unidirectional data channel between processes, returning a pair of file descriptors for the read and write ends as defined in the POSIX specification [4].

A file descriptor can be put into non-blocking mode by setting the O_NONBLOCK flag via fcntl() [4]. In non-blocking mode, read() or write() return immediately without waiting if the operation would block.

The select() system call lets a process monitor multiple file descriptors simultaneously to determine which are ready for I/O operations, enabling concurrent reads from multiple pipes without blocking on any single one. It is defined in the POSIX standard and commonly used for multiplexed I/O [5].

PART B (30 pts): DEVELOPING A MULTITHREADED APPLICATION

Write another program that implements the same project in part A using multiple threads instead of multiple processes [6, 7]. Rather than creating N child processes, the program will create N threads, with one thread assigned to each input file. Each thread processes a distinct input file. Pipes will not be used in this case. Threads can share data by use of global variables.

A thread reads its assigned input file and records the required word information in a data structure. This data structure will be processed by the main (parent) thread after the worker thread has completed execution.

After all threads have terminated, the main thread combines the results produced by the N threads and writes the merged results to the output file in sorted order (as in part A).

In this version, the program will be named **findlwt**, where “t” denotes threads (the C source file will be findlwt.c). It will be invoked with the following command-line parameters.

findlwt FilePrefix N K OutFilename

Example invocations are shown below.

```
./findlwt inputfile 8 12 output
./findlwt infile 3 6 out
```

PART C (20 pts): EXPERIMENTS

You will design and conduct some performance (time) experiments. You will then interpret them. You will put your results, interpretations, and conclusions in a **report.pdf** file. You will include this file as part of your uploaded package. Experiments and the related report will be **20%** of the overall project grade.

You will think and decide what experiments you will perform. For example, you can run your program with different number of child processes (threads), different DataLen values, different K values, and input file sizes, etc. Measure the time elapsed. See if there are differences among results associated with different parameter values. Put your results into tables (or plots). Try to explain the results, trends, and the reasons. Put all your experimental data, analysis and interpretations into the report file.

SUBMISSION

Put all your files into a directory named with your Student ID. If the project is done as a group, the IDs of all students will be written, separated by a dash '-'. In a README.txt file, write your name, ID, etc. (if done as a group, all names and IDs will be included). The set of files in the directory will include README.txt, Makefile, and program source file(s). We should be able to compile your program(s) by just typing make. No binary files (executable files or .o files) will be included in your submission. Then **tar** and **gzip** the directory, and submit it to **Moodle**.

For example, a project group with student IDs 21404312 and 214052104 will create a directory named 21404312-214052104 and will put their files there. Then, they will tar the directory (package the directory) as follows:

```
tar cvf 21404312-214052104.tar 21404312-214052104
```

Then they will gzip (compress) the tar file as follows:

```
gzip 21404312-214052104.tar
```

In this way they will obtain a file called 21404312-214052104.tar.gz. Then they will upload this file to **Moodle**. For a project done individually, just the ID of the student will be used as a file or directory name.

LATE POLICY FOR THE PROJECT

If the deadline is missed by up to 24 hours, 25 points will be cut from the graded project and that will be the grade of the project. For example, if a late submission is graded and received 80 pts; then the grade of the project will be $80 - 25 = 55$. If the deadline is missed between 24 - 48 hours, then 50 pts will be cut. Submission beyond 2 days (48 hours) will not be allowed. Note that even the submission is 1 second later after the deadline, 25 points will be cut. System assigns timestamps to the submissions. We will use those timestamps. Therefore, you are recommended to finish your submission 2-3 hours (at the latest) before the deadline. In fact, it is better if you submit one day before. You can re-submit as many times as you wish by the deadline. Each submission will overwrite the previous one. Please do not send your submission with email. Submission system will be open beyond the deadline up to 48 hours.

TIPS AND CLARIFICATIONS

- Start early, work incrementally.

REFERENCES

- [1]. CS342, "Academic Integrity Policy," 2026. Available: <https://www.cs.bilkent.edu.tr/~korpe/courses/cs342fall2025/integrity.html>
- [2]. A. Silberschatz et al., Operating System Concepts, 10th Edition (Global Edition), Wiley, 2021.
- [3]. Fork system call. <https://man7.org/linux/man-pages/man2/fork.2.html>
- [4]. Pipes. <https://pubs.opengroup.org/onlinepubs/009604599/functions/pipe.html>
- [5]. Select function. <https://man7.org/linux/man-pages/man2/select.2.html>
- [6]. POSIX threads (pthreads). <https://man7.org/linux/man-pages/man7/pthreads.7.html>
- [7]. Wikipedia, "Pthreads,". <https://en.wikipedia.org/wiki/Pthreads>